Datta Meghe College of Engineering Airoli, Navi Mumbai	**DEPARTMENT OF COMPUTER ENGINEERING** **ACADEMIC YEAR: 2025 – 26 (TERM – II)** **List of Experiments** **Course Name: Artificial Intelligence** **Course Code: CSC604**

Sr. No	Name of experiment	Cos Covered	Page No.	Date of Performance	Date of Submission	Marks & Signature
1	**Case study on AI applications published in IEEE/ACM/Springer or any prominent journal.**	CO1				
2	**PEAS representation on AI application.**	CO2				
3	**To implement Breadth-First Search and Depth-First Search**	CO3				
4	**To implement A* and AO* informed Search algorithm.**	CO3				
5	**To implement the N-Queen Algorithm.**	CO3				
6	**To implement First-Order Logic examples**	CO4				
7	**To implement building block game using planning**					
8	**To implement Bayes Belief Network**					
	Assignment 1	All				
	Assignment 2	All				

This is to certify that Mr. / Miss _____ of _____ Roll No. _____ has performed the Experiments / Assignments / Tutorials / Case Study Work mentioned above in the premises of the institution.

_____

Practical Incharge

DATTA MEGHE COLLEGE OF ENGINEERING, AIROLI, NAVI MUMBAI

DEPARTMENT OF COMPUTER ENGINEERING

Institute Vision : To create value - based technocrats to fit in the world of work and research

Institute Mission : • To adopt the best engineering practices

• To empower students to work in the world of technology and research

• To create competent human beings

Department Vision : To provide an intellectually stimulating environment for education, technological excellence in computer engineering field and professional training along with human values.

Department Mission:

M1: To promote an educational environment that combines academics with intellectual curiosity.

M2: To develop human resources with sound knowledge of theory and practical in the discipline of Computer Engineering and the ability to apply the knowledge to the benefit of society at large.

M3: To assimilate creative research and new technologies to facilitate students to be a lifelong learner who will contribute positively to the economic well-being of the nation.

Program Educational Objectives (PEO):

PEO1: To explicate optimal solutions through application of innovative computer science techniques that aid towards betterment of society.

PEO2: To adapt recent emerging technologies for enhancing their career opportunity prospects.

PEO3: To effectively communicate and collaborate as a member or leader in a team to manage multidisciplinary projects.

PEO4: To prepare graduates to become involved in research, higher studies or to become entrepreneurs in the long run.

Program Specific Outcomes (PSO):

PSO1: To apply basic and advanced computational and logical skills to provide solutions to computer engineering problems.

PSO2: Ability to apply standard practices and strategies in design and development of software and hardware-based systems and adapt to evolutionary changes in computing to meet the challenges of the future.

PSO3: To develop an approach for lifelong learning and utilize multi-disciplinary knowledge required for satisfying industry or global requirements.

Program Outcomes as defined by NBA (PO)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **Engineer and society:** Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.
7. **Environment and sustainability:** Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commitment to professional ethics and responsibilities and norms of engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, making effective presentations, and giving and receiving clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

DATTA MEGHE COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COURSE NAME: MOBILE COMPUTING LAB (R-19)

COURSE CODE: CSL603/CSC603

YEAR OF STUDY: T.E., SEMESTER: VI

COURSE OUTCOMES

CSC604.1	Ability to develop a basic understanding of AI building blocks presented in intelligent agents.
CSC604.2	Ability to choose an appropriate problem solving method and knowledge representation technique.
CSC604.3	Ability to analyze strengths and weaknesses of Ai approaches to knowledge-intensive problem solving.
CSC604.4	Ability to design models for reasoning with uncertainty as well as the use of unreliable information.
CSC604.5	Ability to design models for reasoning with uncertainty as well as the use of unreliable information.

DATTA MEGHE COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2024-25 (TERM II)

SUBJECT: ARTIFICIAL INTELLIGENCE

SEM: VI

RUBRICS FOR GRADING EXPERIMENTS

Rubric Number	Rubric Title	Criteria	Marks (out of 15)
R1	Punctuality, Completion Time / Timeline	On-time	3
		Delayed by not more than a week	2
		Delayed more than a week	1
R2	Implementation	Structured program, Correct logic	5
		Partial structured program	4
		Implementation with error	3
		Unacceptable	2-1
R3	Clarity of Concepts	Clear understanding	5
		Partial understanding	4
		Weak understanding	3
		Unacceptable	2-1
R4	Documentation	Correct documentation	2
		Moderate documented	1

DATTA MEGHE COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

ACADEMIC YEAR 2024-25 (TERM II)

SUBJECT:ARTIFICIAL INTELLIGENCE

SEM: VI

RUBRICS FOR GRADING ASSIGNMENTS

Rubric Number	Rubric Title	Criteria	Marks (out of 5)
R1	Punctuality, Completion Time / Timeline	On-time	2
		Delayed by not more than a week	1
		Delayed more than a week	0
R2	Knowledge & Concept	Clear understanding	2
		Partially understood	1
		Weak understanding	0
R3	Documentation	Correct documentation	1
		Not documented properly	0

EXPERIMENT NO :- 01

Date of Performance:
Date of Submission:

Aim: One case study on AI applications published in IEEE/ACM/Springer or any prominent journal.

Theory: **IoT Based Health Monitoring System with AI Powered Disease Prediction**
Publication Year: 2025

Abstract:

In recent years, technologies like Artificial Intelligence (AI) and the Internet of Things (IoT) have started playing an important role in improving healthcare systems. In many cases, patients need to visit hospitals regularly just to monitor basic health conditions, which can be time-consuming and sometimes difficult for people living in remote areas. Because of this, researchers are exploring smart healthcare systems that can monitor patients remotely. One such approach is an IoT-based health monitoring system combined with AI for disease prediction.

In this system, different sensors are used to collect important health data from patients, such as body temperature, heart rate, or oxygen level. These sensors send the collected data to a cloud platform where it can be stored and analyzed. Machine learning algorithms are then used to study this data and identify patterns that may indicate possible health problems. Based on this analysis, the system can predict certain diseases or health risks at an early stage and provide alerts or recommendations.

The main idea behind this approach is to make healthcare monitoring more efficient and accessible. It can help doctors monitor patients remotely and also allow patients to keep track of their health more easily. Overall, combining IoT devices with AI techniques can help in early disease detection, continuous health monitoring, and better management of healthcare services. This study shows that such systems have good potential for improving modern healthcare, especially in situations where regular hospital visits are not always possible.

Conclusion: Thus, We have Studied case study on AI applications published in IEEE/ACM/Springer or any prominent journal.

Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	TOTAL (10 Marks)	Signature

EXPERIMENT NO :- 02

Date of Performance:
Date of Submission:

Aim: Assignments on State-space formulation and PEAS representation for various AI applications

Theory:

PEAS stands for Performance measures, Environment, Actuators, and Sensors. We shall see what these terms mean individually.

Performance measures:

These are the parameters used to measure the performance of the agent. It measures how well the agent is carrying out a particular assigned task.

Environment:

Environment refers to everything that surrounds the agent and within which the agent operates. It includes all external conditions, objects, and situations that the agent can interact with or respond to while performing its task.

Actuators:

Actuators are the components through which an agent takes action in the environment. They allow the agent to perform operations or produce outputs that can affect or change the environment.

Sensors:

Sensors are the components that help the agent perceive or gather information from the environment. They collect data about the surroundings and provide this information to the agent so that it can make decisions and take appropriate actions.

Example:

PEAS (Performance measure, Environment, Actuators, Sensors) for a vacuum cleaner in AI would involve measuring cleaning efficiency (performance), the home environment with obstacles (environment), motors and brushes for movement (actuators), and sensors for detecting dirt and navigation (sensors)

PEAS Components:

PEAS stands for Performance measure, Environment, Actuators, and Sensors, which are components used in defining an intelligent system. Here are the PEAS components for the AI system described in the case study of the paper titled **"IoT Based Health Monitoring System with AI-Powered Disease Prediction."** PEAS for this system includes accurate disease prediction as the performance measure, patient healthcare environment, alerts and recommendations as actuators, and IoT health sensors as sensors.

Performance Measures:

- **Accuracy:** The system should correctly predict diseases based on patient health data.
- **Early Detection:** The system should identify potential health problems at an early stage.
- **Reliability:** The monitoring system should consistently provide correct and stable results.
- **Response time:** The system should quickly analyze data and generate alerts.
- **Reduced hospital visits:** The system should help patients monitor health remotely and reduce frequent hospital trips.

Environment:

- **Patients:** Individuals whose health data is being monitored.
- **Healthcare facilities:** Hospitals and clinics where doctors review patient data.
- **Home monitoring setup:** Patients using wearable or IoT devices at home.
- **Cloud servers:** Platforms where health data is stored and processed.
- **Internet/IoT network:** Communication network connecting sensors, devices, and servers.

Actuators:

- **Alert notifications:** The system sends warnings to patients or doctors when abnormal conditions are detected.
- **Health reports:** The system displays analyzed health data on mobile or web applications.
- **Medical recommendations:** The system suggests possible precautions or actions based on analysis.
- **Emergency alerts:** The system can notify doctors or caregivers in critical situations.

Sensors:

- **Heart rate sensor:** Measures the patient's heart rate continuously.
- **Temperature sensor:** Measures body temperature.
- **Blood pressure sensor:** Monitors the patient's blood pressure levels.
- **Oxygen sensor (SpO_2):** Measures oxygen saturation in the blood.
- **Wearable health devices:** Smart devices used to collect real-time health data.

<u>Conclusion:</u> Thus, We have Studied case study on AI applications published in IEEE/ACM/Springer or any prominent journal.

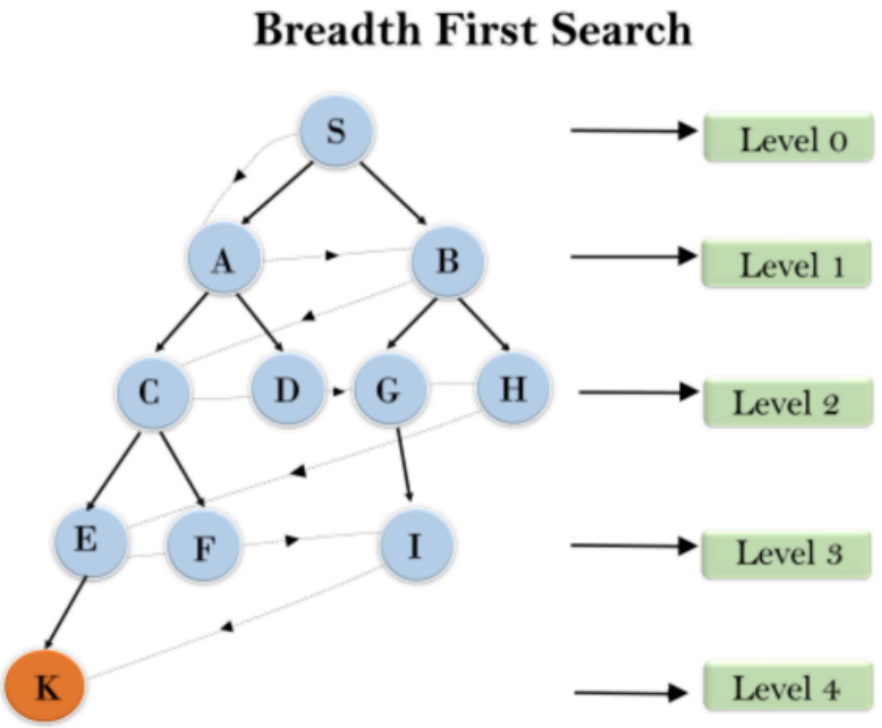
Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	TOTAL (10 Marks)	Signature

EXPERIMENT NO :- 03

Date of Performance:
Date of Submission:

Aim: To implement Breadth First Search as Uniformed Search
Theory:
✔ Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
✔ BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to the next level.
✔ The breadth-first search algorithm is an example of a general-graph search algorithm.
✔ Breadth-first search implemented using FIFO queue data structure.



Algorithm:

The steps of the algorithm work as follow:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+.....+b^d = O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is O(bd).

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

<u>BFS Pseudocode:</u>

The pseudocode for BFS in python goes as below:

```
create a queue Q
mark v as visited and put v into Q
while Q is non-empty
        remove the head u of Q
        mark and enqueue all (unvisited) neighbors of u
```

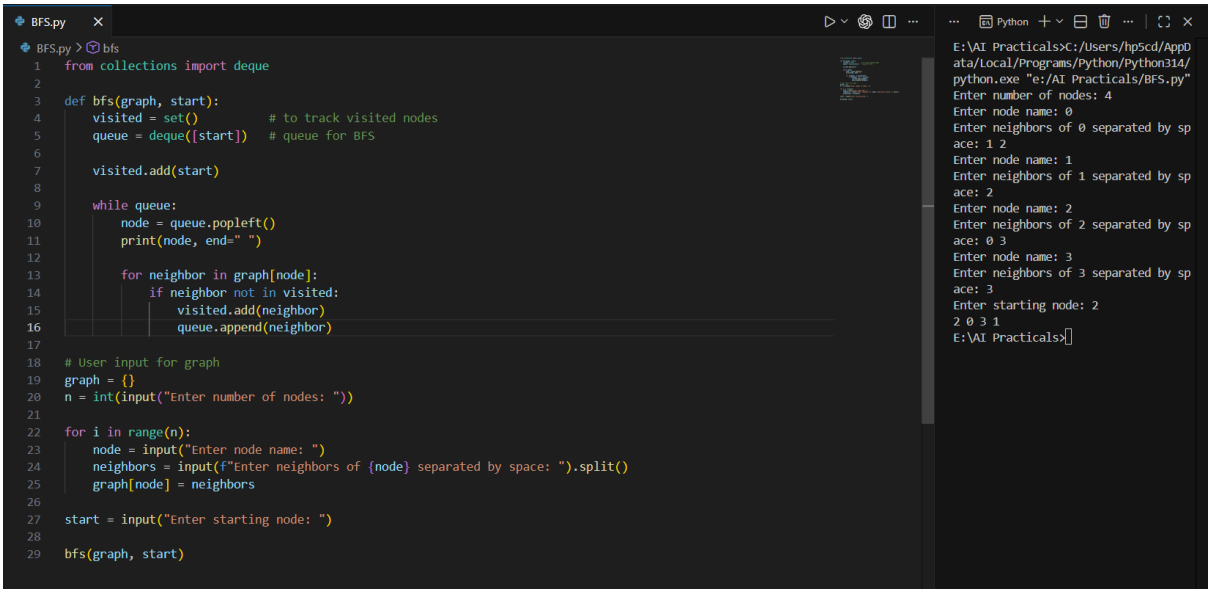
<u>Program:</u>

```
from collections import deque

def bfs(graph, start):
    visited = set()          # to track visited nodes
    queue = deque([start])   # queue for BFS

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# User input for graph
graph = {}
n = int(input("Enter number of nodes: "))
```

```
for i in range(n):
    node = input("Enter node name: ")
    neighbors = input(f"Enter neighbors of {node} separated by space: ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

bfs(graph, start)
```

Output:

Enter number of nodes: 4
Enter node name: 0
Enter neighbors of 0 separated by space: 1 2
Enter node name: 1
Enter neighbors of 1 separated by space: 2
Enter node name: 2
Enter neighbors of 2 separated by space: 0 3
Enter node name: 3
Enter neighbors of 3 separated by space: 3
Enter starting node: 2
2 0 3 1



<u>Conclusion:</u> Thus, We have Studied , implement Breadth First Search as Uniformed Search

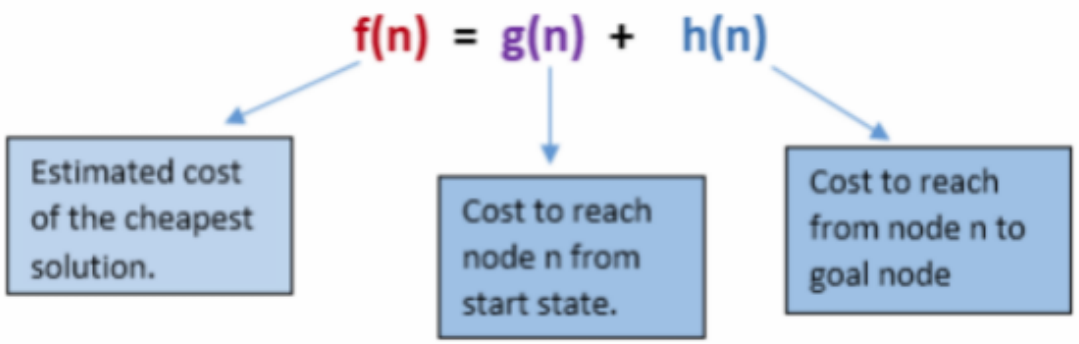
Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	TOTAL (10 Marks)	Signature

EXPERIMENT NO :- 04

Date of Performance:
Date of Submission:

Aim: To implement A* Informed Search Algorithm.
Theory:
A* search is the most commonly known form of best-first search. It uses the heuristic function h(n), and costs to reach the node n from the start state g(n). It has combined features of UCS and greedy best-first search, by which it solves the problem efficiently. A* search algorithm finds the shortest path through the search space using the heuristic function. This search algorithm expands the search tree and provides optimal results faster. A* algorithm is similar to UCS except that it uses g(n)+h(n) instead of g(n).
In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as follows, and this sum is called a fitness number.



Algorithm of A* search:

Step1: Place the starting node in the OPEN list.
Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stop.
Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise
Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute the evaluation function for n' and place it into the Open list.
Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.
Step 6: Return to Step 2.

Advantages:

1. A* search algorithm is the best algorithm than other search algorithms.
2. A* search algorithm is optimal and complete.
3. This algorithm can solve very complex problems.

Disadvantages:

1. It does not always produce the shortest path as it mostly based on heuristics and approximation.
2. A* search algorithm has some complexity issues.
3. The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

Source Code:

```
import heapq

def astar(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        current_cost, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor, cost in graph[current]:
            new_g = g_cost[current] + cost

            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g
                f_cost = new_g + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                parent[neighbor] = current

    return "Path not found"
```

```
# Example Graph (node : [(neighbor, cost)])
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

# Heuristic values (estimated distance to goal)
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 3,
    'E': 1,
    'F': 0
}

start = 'A'
goal = 'F'

path = astar(graph, heuristic, start, goal)
print("Shortest Path:", path)
```

Output:

Shortest Path: ['A', 'B', 'E', 'F']

Astar.py

```python
import heapq

def astar(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {start: 0}
    parent = {start: None}

    while open_list:
        current_cost, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            return path[::-1]

        for neighbor, cost in graph[current]:
            new_g = g_cost[current] + cost

            if neighbor not in g_cost or new_g < g_cost[neighbor]:
                g_cost[neighbor] = new_g
                f_cost = new_g + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
                parent[neighbor] = current

    return "Path not found"


# Example Graph (node : [(neighbor, cost)])
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
```

```
E:\AI Practicals>C:/Users/hp5cd/AppData/Local/Programs/Python/Python314/python.exe "e:/AI Practicals/Astar.py"
Shortest Path: ['A', 'B', 'E', 'F']

E:\AI Practicals>
```

Astar.py

```python


# Example Graph (node : [(neighbor, cost)])
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

# Heuristic values (estimated distance to goal)
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 3,
    'E': 1,
    'F': 0
}

start = 'A'
goal = 'F'

path = astar(graph, heuristic, start, goal)

print("Shortest Path:", path)
```

```
E:\AI Practicals>C:/Users/hp5cd/AppData/Local/Programs/Python/Python314/python.exe "e:/AI Practicals/Astar.py"
Shortest Path: ['A', 'B', 'E', 'F']

E:\AI Practicals>
```

Conclusion: Thus, We have Studied , A* Informed Search Algorithm.

Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	TOTAL (10 Marks)	Signature

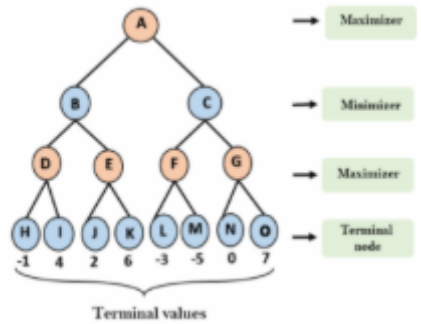
EXPERIMENT NO :- 05

Date of Performance:
Date of Submission:

Aim: Program on Game playing algorithm.
Theory:
Mini-Max Algorithm in Artificial Intelligence
o Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that the opponent is also playing optimally.
o Mini-Max algorithm uses recursion to search through the game-tree.
o Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various two-player games. This Algorithm computes the minimax decision for the current state.
o In this algorithm two players play the game, one is called MAX and other is called MIN.
o Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
o Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
o The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
o The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

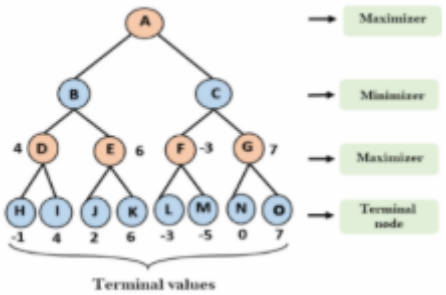
Working of Min-Max Algorithm:
o The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.
o In this example, there are two players one is called Maximizer and other is called Minimizer.
o Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
o This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.
o At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A as the initial state of the tree. Suppose maximizer takes the first turn which has worst-case initial value =- infinity, and minimizer will take next turn which has worst-case initial value = +infinity.



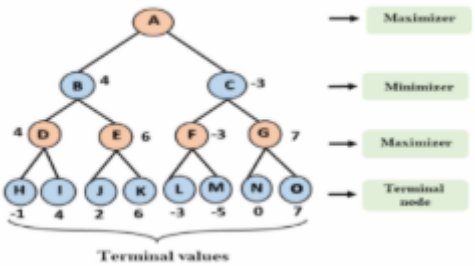
Step 2: Now, first we find the utility value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with the initial value of Maximizer and determine the higher nodes values. It will find the maximum among them all.

- For node D max(-1,- -∞) => max(-1,4)= 4
- For Node E max(2, -∞) => max(2, 6)= 6
- For Node F max(-3, -∞) => max(-3,-5) = -3
- For node G max(0, -∞) = max(0, 7) = 7



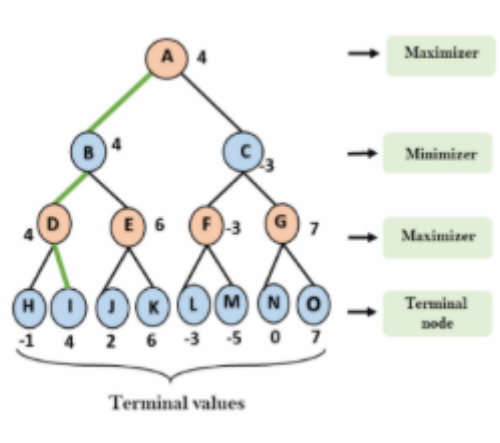
Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with +∞, and will find the 3rd layer node values.

- For node B= min(4,6) = 4
- For node C= min (-3, 7) = -3



Step 4: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes values and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A max(4, -3)= 4



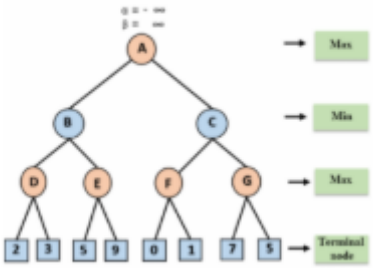
That was the complete workflow of the minimax two player game.

Alpha-Beta Pruning

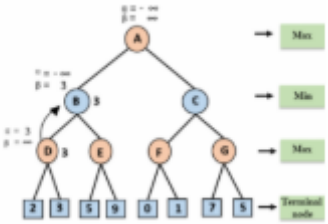
- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm, the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called pruning. This involves two threshold parameters Alpha and beta for future expansion, so it is called alpha-beta pruning. It is also called the Alpha-Beta Algorithm.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prunes the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
    1. **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is -∞.
    2. **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞. o The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making the algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Working of Alpha-Beta Pruning:
Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

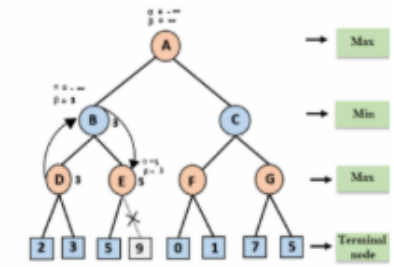
Step 1: At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.



Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and
node value will also 3.
Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.

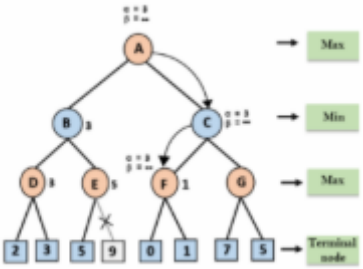


In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5

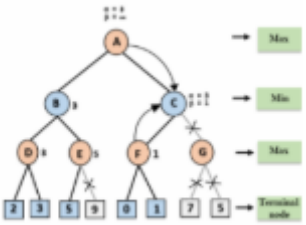


.

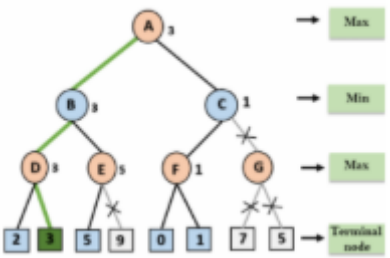
Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed; the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now pass to the right successor of A which is Node C.
At node C, α=3 and β= +∞, and the same values will be passed on to node F.
Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3.
Following is the final game tree which shows the nodes which are computed and nodes which have never been computed. Hence the optimal value for the maximizer is 3 for this example.



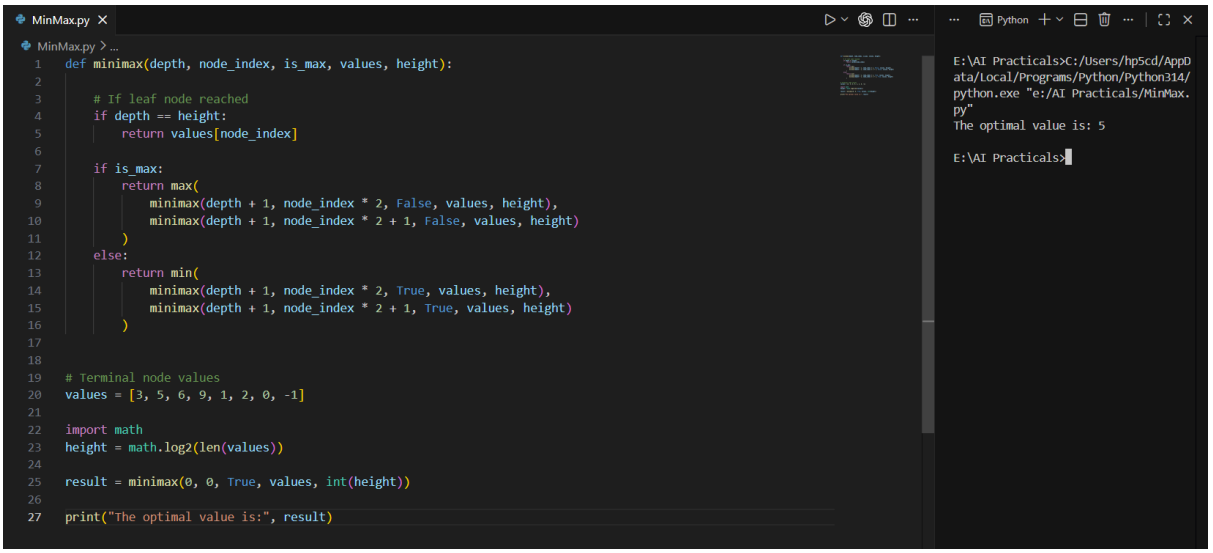
<u>Program:</u>

```
def minimax(depth, node_index, is_max, values, height):

    # If leaf node reached
    if depth == height:
        return values[node_index]

    if is_max:
```

```
        return max(
            minimax(depth + 1, node_index * 2, False, values, height),
            minimax(depth + 1, node_index * 2 + 1, False, values, height)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, values, height),
            minimax(depth + 1, node_index * 2 + 1, True, values, height)
        )


# Terminal node values
values = [3, 5, 6, 9, 1, 2, 0, -1]

import math
height = math.log2(len(values))

result = minimax(0, 0, True, values, int(height))

print("The optimal value is:", result)
```

Output:

The optimal value is: 5



Conclusion: Thus, We have Studied , Program on Game playing algorithm.

Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	TOTAL (10 Marks)	Signature

EXPERIMENT NO. 6

Date of Performance:
Date of Submission:

Aim: Program for first order Logic.(Mini Project)

Software Used: Prolog

Theory:

Knowledge engineering is the base that helped in the creation of expert systems where knowledge is transformed into computer programs. Expert systems have a huge and flexible knowledge base which is integrated with mechanisms that specify how to use the information of the knowledge base and apply it to a variety of situations. These expert systems also use machine learning and deep learning algorithms in order to learn as humans do. Nowadays these Expert systems are used in the education field, healthcare, financial services, manufacturing, etc.

Process of Knowledge Engineering

Knowledge Engineering for different domains is different but it follows, the same set of rules/procedures in order to create expert systems.

1. Task Identification

This is the first initial stage where the task to be performed is defined. In a domain, a specific problem or a combination of several problems would be taken. This task must be realistic and the subject matter expert shall have a clear picture of what it is so that further process can be carried out.

2. Acquisition of Knowledge

Once the problem is well defined then the next step is to gather relevant knowledge and information about the problem. For some problems standard data is used that must be collected, for example, a problem on heat exchanger requires the standard steam table data at x temperature and y pressure what will be the value of enthalpy.

3. Prepare a road map

Once the goal and knowledge base are available the next step is to get the roadmap ready by breaking the goal down into small steps by questionnaires and relevant knowledge base. Here subject matter expert puts his thoughts on how would he make decisions and what parameters would be considered at all stages. There could be several ways to solve some problems, and all should be considered.

4. Encode

Now it's time to convert this knowledge into computer language. Here the knowledge is encoded by using different functions as well as in some cases, for a specific task, the algorithm is used to

create a model. These models are able to make decisions based on available parameters as an expert does, surely the model must be trained and tested on a sufficient amount of data.

5. Evaluation and Debugging

In the process of creating an expert system, at each step, the model should be evaluated and debugged and then added to workflow. Once all small tasks are evaluated, they are assembled to create one whole expert system. This system is again evaluated on similar problems and Debugged if any issue is there.

6. Justification & Explanation

Here the model is justified for the given task and working is explained.

First-Order Logic in Artificial intelligence

In the topic of Propositional logic, we have seen that how to represent statements using propositional logic. But unfortunately, in propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent the complex sentences or natural language statements. The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.

- **"Some humans are intelligent", or**
- **"Sachin likes cricket."**

To represent the above statements, PL logic is not sufficient, so we required some more powerful logic, such as first-order logic.

First-Order logic:

- First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- First-order logic is also known as **Predicate logic or First-order predicate logic**. First-order logic is a powerful language that develops information about the objects in a more easy way and can also express the relationship between those objects.
- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
    - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, ...
    - **Relations: It can be unary relation such as:** red, round, is adjacent, **or n-any relation such as:** the sister of, brother of, has color, comes between
    - **Function:** Father of, best friend, third inning of, end of, ...
- As a natural language, first-order logic also has two main parts:

    - **Syntax**

- **Semantics**

Syntax of First-Order logic:

The syntax of FOL determines which collection of symbols is a logical expression in first-order logic. The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.

Basic Elements of First-order logic:

Following are the basic elements of FOL syntax:

Constant	1, 2, A, John, Mumbai, cat,....
Variables	x, y, z, a, b,....
Predicates	Brother, Father, >,....
Function	sqrt, LeftLegOf, ...
Connectives	∧, ∨, ¬, ⇒, ⇔
Equality	==
Quantifier	∀, ∃

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as **Predicate (term1, term2, ..., term n)**.

Example: Ravi and Ajay are brothers: => Brothers(Ravi, Ajay).
Chinky is a cat: => cat (Chinky).

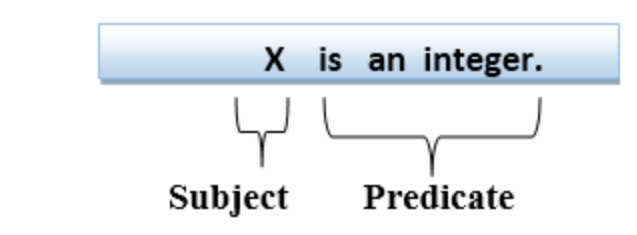
Complex Sentences:

- Complex sentences are made by combining atomic sentences using connectives.

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer.", it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Quantifiers in First-order logic:

- A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.
- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:

1. **Universal Quantifier, (for all, everyone, everything)**
2. **Existential quantifier, (for some, at least one).**

Universal Quantifier:

Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.

The Universal quantifier is represented by a symbol ∀, which resembles an inverted A.

Note: In universal quantifier we use implication "→".

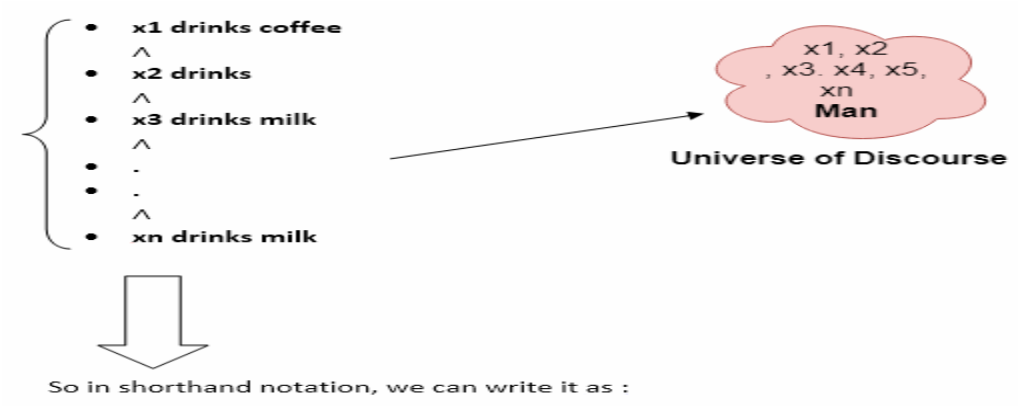
If x is a variable, then ∀x is read as:

- **For all x**
- **For each x**
- **For every x.**

Example:

All man drink coffee.

Let a variable x which refers to a cat so all x can be represented in UOD as below:



∀x man(x) → drink (x, coffee).

It will be read as: There are all x where x is a man who drink coffee.

Existential Quantifier:

Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.

It is denoted by the logical operator ∃, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.

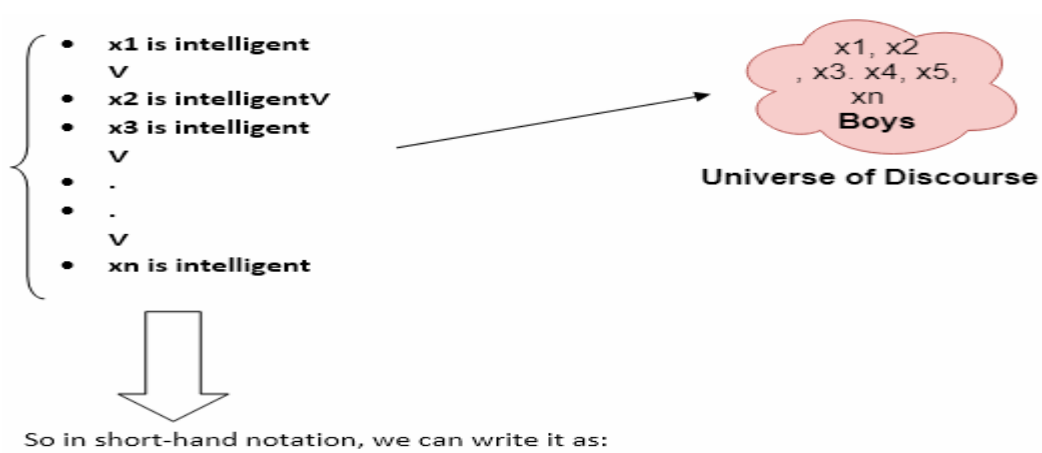
> *Note: In Existential quantifier we always use AND or Conjunction symbol (∧).*

If x is a variable, then existential quantifier will be ∃x or ∃(x). And it will be read as:

- **There exists a 'x.'**
- **For some 'x.'**
- **For at least one 'x.'**

Example:

Some boys are intelligent.



∃x: boys(x) ∧ intelligent(x)

It will be read as: There are some x where x is a boy who is intelligent.

Points to remember:

- The main connective for universal quantifier ∀ is implication →.
- The main connective for existential quantifier ∃ is and ∧.

Properties of Quantifiers:

- In universal quantifier, ∀x∀y is similar to ∀y∀x.
- In Existential quantifier, ∃x∃y is similar to ∃y∃x.
- ∃x∀y is not similar to ∀y∃x.

Some Examples of FOL using quantifier:

1. All birds fly.
In this question the predicate is "**fly(bird)**."
And since there are all birds who fly so it will be represented as follows.

∀x bird(x) →fly(x).

2. Every man respects his parent.
In this question, the predicate is "**respect(x, y)," where x=man, and y= parent**.
Since there is every man so will use ∀, and it will be represented as follows:

∀x man(x) → respects (x, parent).

3. Some boys play cricket.
In this question, the predicate is "**play(x, y)**," where x= boys, and y= game. Since there are some boys so we will use **∃, and it will be represented as**:

∃x boys(x) → play(x, cricket).

4. Not all students like both Mathematics and Science.
In this question, the predicate is "**like(x, y)," where x= student, and y= subject**.
Since there are not all students, so we will use **∀ with negation, so** following representation for this:

¬∀ (x) [student(x) → like(x, Mathematics) **∧ like(x, Science)].**

5. Only one student failed in Mathematics.
In this question, the predicate is "**failed(x, y)," where x= student, and y= subject**.
Since there is only one student who failed in Mathematics, so we will use following representation for this:

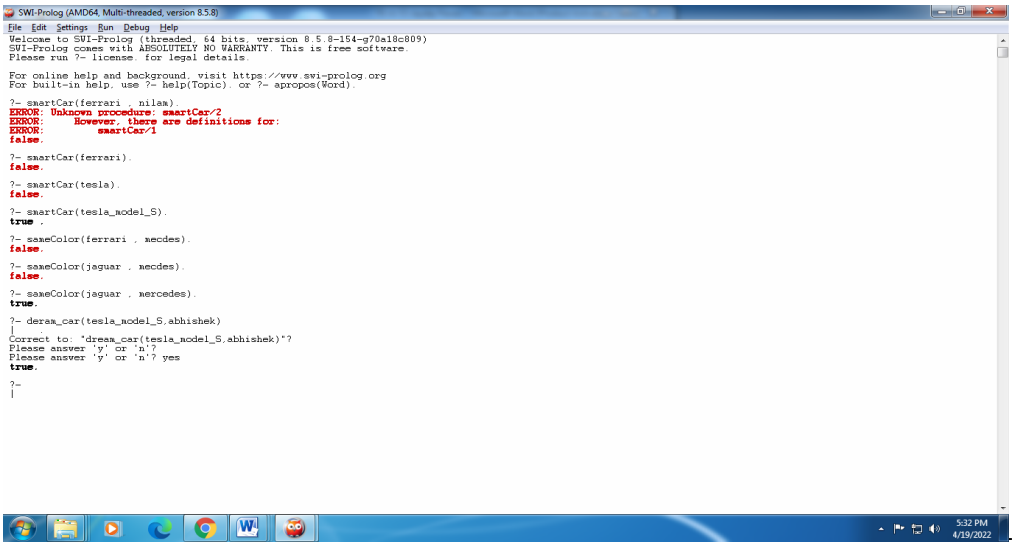
∃(x) [student(x) → failed (x, Mathematics) ∧∀ (y) [¬(x==y) ∧ student(y) → ¬failed (x, Mathematics)]

Program:

```
main:-write('Car').

car(ferrari).
car(jaguar).
car(bmw).
car(mercedes).
car(tesla_model_S).

car_color(ferrari,red).
car_color(jaguar,black).
car_color(bmw,white).
car_color(mercedes,black).
car_color(tesla_model_S,white).

brand(bmw , bmw ).
brand(mercedes , mercedes).
brand(tata,jaguar).

name(nilam).
name(pravin).
name(manasi).
name(chirag).
name(abhishek).
name(tejas).

loveCar(ferrari,nilam).
loveCar(jaguar,pravin).
loveCar(mercedes,chirag).
loveCar(bmw,manasi).
loveCar(tesla_model_S,abhishek).
raceCarSpeed(400).

fuel(mercedes,petrol).
fuel(bmw,disel).
fuel(jaguar,cng).
fuel(ferrari,steam).
fuel(electric,tesla_model_S).

%km/hr
milage(ferrari,200).
milage(mercedes,300).
milage(bmw,300).
milage(jaguar,400).
```

```
featureOfCar(tesla_model_S ,automate ).
featureOfCar(bmw , smart_door ).
featureOfCar(mercedes , smart_balance).
featureOfCar(ferrari , speed).

dream_car(X , Y):-
   loveCar(X , Y),name(Y).

likeRacing(X):-
   name(X),loveCar(ferrari ,X)|loveCar(jaguar , X).

sameColor(X , Y ):-
   car_color(X , Z),car_color(Y , Z).

sameMilage(A , B):-
   milage(A , C),milage(B ,C).

racing(X):-
   milage(X , Z),raceCarSpeed(Z) , car(X).

fuelEfficientCar(X):-
   fuel(X,disel)| fuel(X , cng).

likeBrand(X ,Y):-
   loveCar(A ,X),brand(A , Y).
smartCar(X):-
   featureOfCar(X , automate)|featureOfCar(X , smart_door)|featureOfCar(X , smart_balance).

iscar(X,Y):-
   car(X),car_color(X , Y).
```

Output:



```
SWI-Prolog (AMD64, Multi-threaded, version 8.5.8)
File  Edit  Settings  Run  Debug  Help
Welcome to SWI-Prolog (threaded, 64 bits, version 8.5.8-154-g70a18c809)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?- smartCar(ferrari , nilam).
ERROR: Unknown procedure: smartCar/2
ERROR:     However, there are definitions for:
ERROR:         smartCar/1
false.

?- smartCar(ferrari).
false.

?- smartCar(tesla).
false.

?- smartCar(tesla_model_S).
true .

?- sameColor(ferrari , mecdes).
false.

?- sameColor(jaguar , mecdes).
false.

?- sameColor(jaguar , mercedes).
true.

?- deram_car(tesla_model_S,abhishek)
|
Correct to: "dream_car(tesla_model_S,abhishek)"?
Please answer 'y' or 'n'?
Please answer 'y' or 'n'? yes
true.

?-
|
```

Conclusion: Thus, We have Studied , Program on Game playing algorithm.

Sign and Remark:

R1 (2 Marks)	R2 (2 Marks)	R3 (1 Marks)	Total (5 Marks)	Signature

EXPERIMENT NO. 7

Date of Performance:

Date of Submission:

Aim: Planning Programming

Theory:

What is planning in AI?

- The planning in Artificial Intelligence is about the decision making tasks performed by the robots or computer programs to achieve a specific goal.
- The execution of planning is about choosing a sequence of actions with a high likelihood to complete the specific task.

Blocks-World planning problem

- The blocks-world problem is known as **Sussman Anomaly.**
- Noninterleaved planners of the early 1970s were unable to solve this problem, hence it is considered as anomalous.
- When two subgoals G1 and G2 are given, a noninterleaved planner produces either a plan for G1 concatenated with a plan for G2, or vice-versa.
- In blocks-world problem, three blocks labeled as 'A', 'B', 'C' are allowed to rest on the flat surface. The given condition is that only one block can be moved at a time to achieve the goal.
- The start state and goal state are shown in the following diagram.

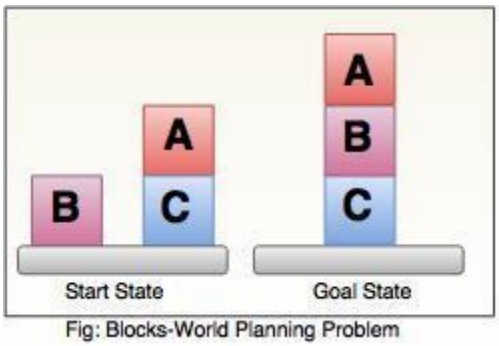


Fig: Blocks-World Planning Problem

Components of Planning System

The planning consists of following important steps:

- Choose the best rule for applying the next rule based on the best available heuristics.
- Apply the chosen rule for computing the new problem state.
- Detect when a solution has been found.
- Detect dead ends so that they can be abandoned and the system's effort is directed in more fruitful directions.
- Detect when an almost correct solution has been found.

Goal stack planning

This is one of the most important planning algorithms, which is specifically used by **STRIPS.**

- The stack is used in an algorithm to hold the action and satisfy the goal. A knowledge base is used to hold the current state, actions.
- Goal stack is similar to a node in a search tree, where the branches are created if there is a choice of an action.

The important steps of the algorithm are as stated below:

i. Start by pushing the original goal on the stack. Repeat this until the stack becomes empty. If stack top is a compound goal, then push its unsatisfied subgoals on the stack.
ii. If stack top is a single unsatisfied goal then, replace it by an action and push the action's precondition on the stack to satisfy the condition.
iii. If stack top is an action, pop it from the stack, execute it and change the knowledge base by the effects of the action.
iv. If stack top is a satisfied goal, pop it from the stack

Program:

```
tab = []

result = []

goalList = ["a", "b", "c", "d", "e"]


def parSolution(N):

   for i in range(N):

      if goalList[i] != result[i]:

         return False

   return True




def Onblock(index, count):


   # break point of recursive call

   if count == len(goalList)+1:

      return True
```

```
    # copy tab of index value to result
    block = tab[index]
    # stack block
    result.append(block)
    print(result)
    if parSolution(count):
        print("Pushed a result solution ")
        # remove block from tab
        tab.remove(block)
        Onblock(0, count + 1)
    else:
        print("result solution not possible, back to the tab")
        # pop out if no partial solution
        result.pop()
        Onblock(index+1, count)



def Ontab(problem):
    # check if everything in stack is on the tab
    if len(problem) != 0:
        tab.append(problem.pop())
        Ontab(problem)
    # if everything is on the tab the we return true
    else:
        return True



def goal_stack_planing(problem):
```

```
    # pop problem and put in tab
    Ontab(problem)
    # print index and number of blocks on result stack
    if Onblock(0, 1):
        print(result)
if __name__ == "__main__":
    problem = ["c", "a", "e", "d", "b"]
    print("Goal Problem")
    for k, j in zip(goalList, problem):
        print(k+"    "+j)
    goal_stack_planing(problem)
    print("result Solution")
    print(result)
```

Output:

```
Goal Problem
a    c
b    a
c    e
d    d
e    b
['b']
result solution not possible, back to the tab
['d']
result solution not possible, back to the tab
['e']
result solution not possible, back to the tab
['a']
Pushed a result solution
['a', 'b']
Pushed a result solution
['a', 'b', 'd']
result solution not possible, back to the tab
['a', 'b', 'e']
result solution not possible, back to the tab
['a', 'b', 'c']
Pushed a result solution
['a', 'b', 'c', 'd']
Pushed a result solution
['a', 'b', 'c', 'd', 'e']
Pushed a result solution
result Solution
['a', 'b', 'c', 'd', 'e']
```

CONCLUSION:

Thus ,We have studied Planning Programming

Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	Total (10 Marks)	Signature

EXPERIMENT NO. 8

Date of Performance:

Date of Submission:

Aim: Implementation for Bayes Belief Network

Theory:

Bayesian Belief Network in artificial intelligence. Bayesian belief network is key computer technology for dealing with probabilistic events and to solve a problem which has uncertainty. We can define a Bayesian network as:

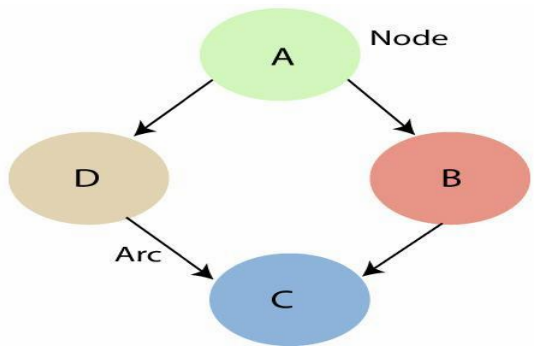
"A Bayesian network is a probabilistic graphical model which represents a set of variables and their conditional dependencies using a directed acyclic graph."

It is also called a **Bayes network, belief network, decision network**, or **Bayesian model**. Bayesian networks are probabilistic, because these networks are built from a **probability distribution**, and also use probability theory for prediction and anomaly detection.

Real world applications are probabilistic in nature, and to represent the relationship between multiple events, we need a Bayesian network. It can also be used in various tasks including **prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction**, and **decision making under uncertainty**.

Bayesian Network can be used for building models from data and experts opinions, and it consists of two parts: **Directed Acyclic Graph, Table of conditional probabilities.**

The generalized form of Bayesian network that represents and solve decision problems under uncertain knowledge is known as an **Influence diagram**. **A Bayesian network graph is made up of nodes and Arcs (directed links), where:**



- Each **node** corresponds to the random variables, and a variable can be **continuous** or **discrete**.
- **Arc or directed arrows** represent the causal relationship or conditional probabilities between random variables. These directed links or arrows connect the pair of nodes in the graph.
  These links represent that one node directly influence the other node, and if there is no directed link that means that nodes are independent with each other
  - **In the above diagram, A, B, C, and D are random variables represented by the nodes of the network graph.**
  - **If we are considering node B, which is connected with node A by a directed arrow, then node A is called the parent of Node B.**

- **Node C is independent of node A.**

The Bayesian network has mainly two components: **Causal Component and Actual numbers**

Each node in the Bayesian network has condition probability distribution $\mathbf{P(X_i \mid Parent(X_i))}$, which determines the effect of the parent on that node. Bayesian network is based on Joint probability distribution and conditional probability. So let's first understand the joint probability distribution:

Joint probability distribution:

If we have variables x1, x2, x3,....., xn, then the probabilities of a different combination of x1, x2, x3.. xn, are known as Joint probability distribution.

$\mathbf{P[x_1, x_2, x_3,....., x_n]}$, it can be written as the following way in terms of the joint probability distribution.

$\mathbf{= P[x_1 \mid x_2, x_3,....., x_n]P[x_2, x_3,....., x_n]}$

$\mathbf{= P[x_1 \mid x_2, x_3,....., x_n]P[x_2 \mid x_3,....., x_n]....P[x_{n-1} \mid x_n]P[x_n].}$

In general for each variable Xi, we can write the equation as:

$$P(X_i \mid X_{i-1},....., X_1) = P(X_i \mid Parents(X_i))$$

Explanation of Bayesian network:

Let's understand the Bayesian network through an example by creating a directed acyclic graph:

Example: Harry installed a new burglar alarm at his home to detect burglary. The alarm reliably responds at detecting a burglary but also responds for minor earthquakes. Harry has two neighbors David and Sophia, who have taken a responsibility to inform Harry at work when they hear the alarm. David always calls Harry when he hears the alarm, but sometimes he got confused with the phone ringing and calls at that time too. On the other hand, Sophia likes to listen to high music, so sometimes she misses to hear the alarm. Here we would like to compute the probability of Burglary Alarm.

Problem:

Calculate the probability that alarm has sounded, but there is neither a burglary, nor an earthquake occurred, and David and Sophia both called the Harry.

Solution:

- The Bayesian network for the above problem is given below. The network structure is showing that burglary and earthquake is the parent node of the alarm and directly affecting the probability of alarm's going off, but David and Sophia's calls depend on alarm probability.
- The network is representing that our assumptions do not directly perceive the burglary and also do not notice the minor earthquake, and they also not confer before calling.
- The conditional distributions for each node are given as conditional probabilities table or CPT.
- Each row in the CPT must be sum to 1 because all the entries in the table represent an exhaustive set of cases for the variable.

- In CPT, a boolean variable with k boolean parents contains 2^K probabilities. Hence, if there are two parents, then CPT will contain 4 probability values

List of all events occurring in this network:

- **Burglary (B)**
- **Earthquake(E)**
- **Alarm(A)**
- **David Calls(D)**
- **Sophia calls(S)**

We can write the events of problem statement in the form of probability: **P[D, S, A, B, E]**, can rewrite the above probability statement using joint probability distribution:

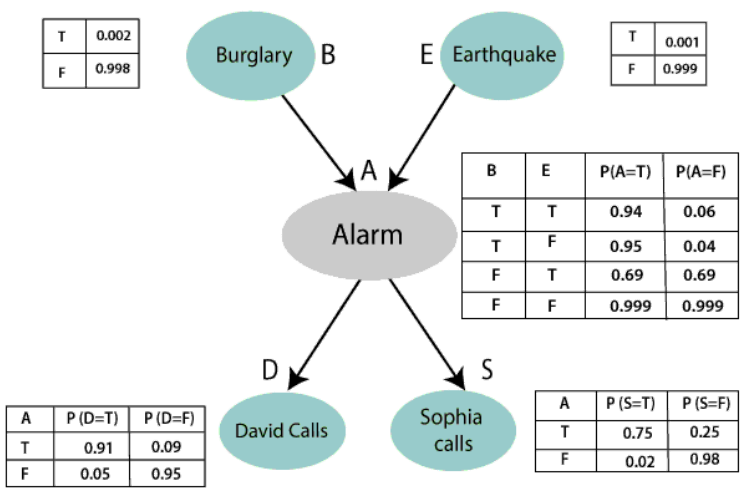
P[D, S, A, B, E]= P[D | S, A, B, E]. P[S, A, B, E]

=P[D | S, A, B, E]. P[S | A, B, E]. P[A, B, E]

= P [D| A]. P [S| A, B, E]. P[A, B, E]

= P[D | A]. P[S | A]. P[A| B, E]. P[B, E]

= P[D | A]. P[S | A]. P[A| B, E]. P[B |E]. P[E]



T	0.002
F	0.998

T	0.001
F	0.999

B	E	P(A=T)	P(A=F)
T	T	0.94	0.06
T	F	0.95	0.04
F	T	0.69	0.69
F	F	0.999	0.999

A	P (D=T)	P (D=F)
T	0.91	0.09
F	0.05	0.95

A	P (S=T)	P (S=F)
T	0.75	0.25
F	0.02	0.98

Let's take the observed probability for the Burglary and earthquake component:

P(B= True) = 0.002, which is the probability of burglary.

P(B= False)= 0.998, which is the probability of no burglary.

P(E= True)= 0.001, which is the probability of a minor earthquake

P(E= False)= 0.999, Which is the probability that an earthquake not occurred.

We can provide the conditional probabilities as per the below tables:

Conditional probability table for Alarm A:

The Conditional probability of Alarm A depends on Burglar and earthquake:

B	E	P(A= True)	P(A= False)
True	True	0.94	0.06
True	False	0.95	0.04
False	True	0.31	0.69
False	False	0.001	0.999

Conditional probability table for David Calls:

The Conditional probability of David that he will call depends on the probability of Alarm.

A	P(D= True)	P(D= False)
True	0.91	0.09
False	0.05	0.95

Conditional probability table for Sophia Calls:

The Conditional probability of Sophia that she calls is depending on its Parent Node "Alarm."

A	P(S= True)	P(S= False)
True	0.75	0.25
False	0.02	0.98

From the formula of joint distribution, we can write the problem statement in the form of probability distribution:

P(S, D, A, ¬B, ¬E) = P (S|A) *P (D|A)*P (A|¬B ^ ¬E) *P (¬B) *P (¬E).

= 0.75* 0.91* 0.001* 0.998*0.999

= 0.00068045.

Hence, a Bayesian network can answer any query about the domain by using Joint distribution.

Program:

```
import math

from pomegranate import *


# Initially the door selected by the guest is completely random

guest =DiscreteDistribution( { 'A': 1./3, 'B': 1./3, 'C': 1./3 } )


# The door containing the prize is also a random process

prize =DiscreteDistribution( { 'A': 1./3, 'B': 1./3, 'C': 1./3 } )


# The door Monty picks, depends on the choice of the guest and the prize door

monty =ConditionalProbabilityTable(

[[ 'A', 'A', 'A', 0.0 ],

[ 'A', 'A', 'B', 0.5 ],

[ 'A', 'A', 'C', 0.5 ],

[ 'A', 'B', 'A', 0.0 ],

[ 'A', 'B', 'B', 0.0 ],

[ 'A', 'B', 'C', 1.0 ],

[ 'A', 'C', 'A', 0.0 ],

[ 'A', 'C', 'B', 1.0 ],

[ 'A', 'C', 'C', 0.0 ],

[ 'B', 'A', 'A', 0.0 ],

[ 'B', 'A', 'B', 0.0 ],

[ 'B', 'A', 'C', 1.0 ],

[ 'B', 'B', 'A', 0.5 ],
```

```
[ 'B', 'B', 'B', 0.0 ],
[ 'B', 'B', 'C', 0.5 ],
[ 'B', 'C', 'A', 1.0 ],
[ 'B', 'C', 'B', 0.0 ],
[ 'B', 'C', 'C', 0.0 ],
[ 'C', 'A', 'A', 0.0 ],
[ 'C', 'A', 'B', 1.0 ],
[ 'C', 'A', 'C', 0.0 ],
[ 'C', 'B', 'A', 1.0 ],
[ 'C', 'B', 'B', 0.0 ],
[ 'C', 'B', 'C', 0.0 ],
[ 'C', 'C', 'A', 0.5 ],
[ 'C', 'C', 'B', 0.5 ],
[ 'C', 'C', 'C', 0.0 ]], [guest, prize] )

d1 = State( guest, name="guest" )
d2 = State( prize, name="prize" )
d3 = State( monty, name="monty" )

#Building the Bayesian Network
network = BayesianNetwork( "Solving the Monty Hall Problem With Bayesian Networks" )
network.add_states(d1, d2, d3)
network.add_edge(d1, d3)
network.add_edge(d2, d3)
network.bake()
```

Output:

```
beliefs = network.predict_proba({'guest' : 'A', 'monty' : 'B'})
print("n".join( "{}t{}".format( state.name, str(belief) ) for state, belief in
```

```
guest A
prize {
"class" :"Distribution",
"dtype" :"str",
"name" :"DiscreteDistribution",
"parameters" :[
{
"A" :0.3333333333333334,
"B" :0.0,
"C" :0.6666666666666664
}
],
}
monty B
```

Conclusion:

Thus We have Studied Implementation for Bayes Belief Network

Sign and Remark:

R1 (2 Marks)	R2 (4 Marks)	R3 (4 Marks)	Total (10 Marks)	Signature